

Question No:2

DS	Session	Tree 1	Question Information	Level 1	Challenge 62
Question description There are three common ways to traverse the nodes of a binary tree: • <i>Preorder</i>: First process the root, then the left subtree, and finally the right subtree. • <i>Inorder</i>: First process the left subtree, then the root, and finally the right subtree. • <i>Postorder</i>: First process the left subtree, then the right subtree, and finally the root. There is a binary tree of n nodes with distinct labels. You are given the preorder and inorder traversals of the tree, and your task is to determine its postorder traversal. Constraints • $1 \leq n \leq 10^5$ Input The first input line has an integer n : the number of nodes. The nodes are numbered $1, 2, \dots, n$. After this, there are two lines describing the preorder and inorder traversals of the tree. Both lines consist of n integers. You can assume that the input corresponds to a binary tree. Output Print the postorder traversal of the tree.					

▼ Logical Test Cases

Test Case 1

INPUT (STDIN)

5
5 3 2 1 4
3 5 1 2 4

EXPECTED OUTPUT

3 1 4 2 5

Test Case 2

INPUT (STDIN)

6
6 5 3 2 1 4
6 3 5 1 2 4

EXPECTED OUTPUT

3 1 4 2 5 6

▼ Mandatory Test Cases

Test Case 1

KEYWORD

for (i=1;i<=n;i++)

▼ Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

8

Test Case 2

TOKEN COUNT

390

Test Case 3

NLOC

41

Code:

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  struct Node {
5      int data;
6      struct Node *left;
7      struct Node *right;
8  };
9
10 struct Node* createNode(int data) {
11     struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
12
13     newNode->data = data;
14     newNode->left = NULL;
15     newNode->right = NULL;
16
17     return newNode;
18 }
19
20 int main() {
21
22     int n;
23     scanf("%d", &n);
24
25     int *preorder = malloc(n * sizeof(int));
26     int *inorder = malloc(n * sizeof(int));
27
28     for (int i = 0; i < n; i++)
29         scanf("%d", &preorder[i]);
```

```
30
31     for (int i = 0; i < n; i++)
32         scanf("%d", &inorder[i]);
33
34     struct Node **stack = malloc(n * sizeof(struct Node*));
35
36     int top = -1;
37     int inIndex = 0;
38
39     struct Node *root = createNode(preorder[0]);
40
41     stack[++top] = root;
42
43     for (int i = 1; i < n; i++) {
44
45         struct Node *node = NULL;
46
47         if (stack[top]->data != inorder[inIndex]) {
48
49             node = createNode(preorder[i]);
50
51             stack[top]->left = node;
52             stack[++top] = node;
53         }
54         else {
55
```

```

56         while (top >= 0 &&
57             stack[top]->data == inorder[inIndex]) {
58
59             node = stack[top--];
60             inIndex++;
61         }
62
63         struct Node *rightNode = createNode(preorder[i]);
64
65         node->right = rightNode;
66         stack[++top] = rightNode;
67     }
68 }
69
70 /* Postorder traversal using two stacks */
71
72 struct Node **s1 = malloc(n * sizeof(struct Node*));
73 struct Node **s2 = malloc(n * sizeof(struct Node*));
74
75 int top1 = -1;
76 int top2 = -1;
77
78 s1[++top1] = root;
79
80 while (top1 >= 0) {
81

```

```
81
82     struct Node *current = s1[top1--];
83
84     s2[++top2] = current;
85
86     if (current->left != NULL)
87         s1[++top1] = current->left;
88
89     if (current->right != NULL)
90         s1[++top1] = current->right;
91 }
92
93 while (top2 >= 0) {
94     printf("%d ", s2[top2--]->data);
95 }
96
97 printf("\n");
98
99 free(preorder);
100 free(inorder);
101 free(stack);
102 free(s1);
103 free(s2);
104
105 return 0;
106 }
```

Output:

Test Case-1:

	5	
	5 3 2 1 4	
	3 5 1 2 4	
	3 1 4 2 5	

Test Case-2:

	6	
	6 5 3 2 1 4	
	6 3 5 1 2 4	
	3 1 4 2 5 6	

Keshav Kumar Rauniyar
CH.SC.U4CSE25236